

FCFD1.2 Reference Manual

Version 0.4

Author: FCFD design team

Date: August 19, 2026

Fermilab

Contents

1	Introduction	1
2	Analog Frontend	2
3	Clock Divider	3
3.1	Operation and Configuration	3
3.2	Test Procedure	3
4	TDC	5
5	I²C Slow Control	10
6	Fast readout	14
6.1	Fast Readout Architecture	14
6.2	Readout data frame	14
6.3	Link test patterns and scrambling	15
7	I²C Readout	16
8	Pinout	18
8.1	pin name and map	18
8.2	Pinout table	18

1 Introduction

Empty now.

2 Analog Frontend

3 Clock Divider

3.1 Operation and Configuration

The `clock_divider` module, shown in Figure 1, generates the main 40 MHz timing signals `clk40` and `tick40_ce` from the 320 MHz reference clock `clk320`. It also generates a dedicated 40 MHz clock, `clk40_i2c`, for the I²C block. In addition, the module produces the bunch-crossing counter `bx_cnt`, which is used by the readout logic.

The phase of `clk40` can be adjusted through the I²C interface. Depending on the selected test-signal mode, `test_signal_input` can be used as an external timing reference for `clk40` phase measurement or for `bx_cnt` alignment.

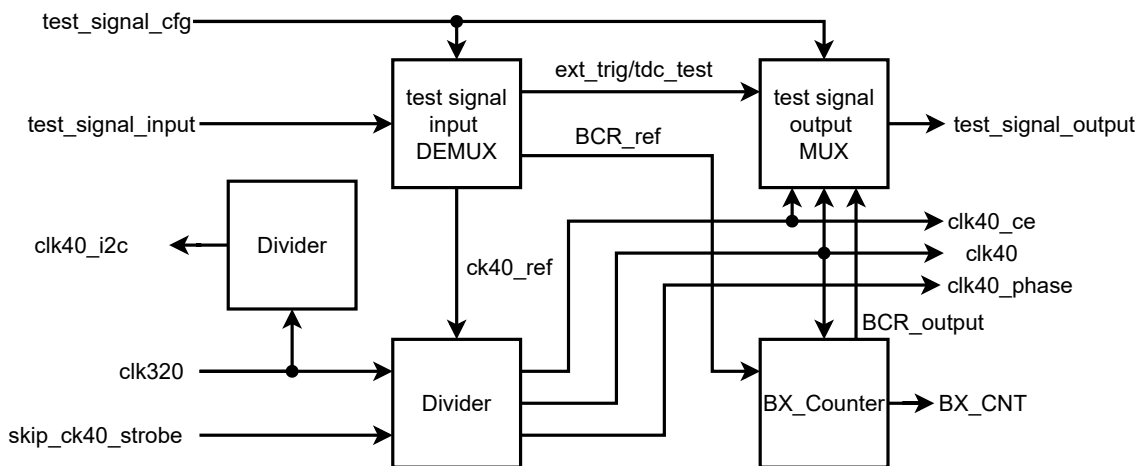


Figure 1: Clock divider block diagram

The primary 320 MHz reference clock, `clk320`, is provided through the clock input. The clock receiver is always on and cannot be disabled. The `clock_divider` module can be reset by the falling edge of the digital reset input on pin 66, as listed in Table 7 in Section 8.

The main 40 MHz clock, `clk40`, can be phase adjusted through the `skip_clk40_strobe` control strobe in the I²C block. To measure its phase relative to an external 40 MHz reference, apply the external clock to the test input port, pins 3 and 4, and set `clock_divider_config` to 2'b01. The relative phase between `clk40` and the external clock is reported by the 3-bit `clk40_phase` value.

The dedicated 40 MHz clock for the I²C block, `clk40_i2c`, is not phase adjustable.

The test input port is also used for bunch-crossing counter alignment. When `clock_divider_config` is set to 2'b10, the input signal is treated as an external BCR input pulse and is used to align the bunch-crossing counter `bx_cnt`. When `clock_divider_config` is set to 2'b00 or 2'b11, the test input signal is ignored by the `clock_divider` module.

The `clock_divider_config` setting also selects the source of `test_signal_output`. As described in Section 5, when `test_out_sel` is set to 0, `test_signal_output` is routed to the chip-level test-output port. The available output selections are summarized in Table 1.

3.2 Test Procedure

The clock divider functions can be verified using I²C commands and an oscilloscope. The relevant I²C registers are `clock_divider_config[1:0]`, `test_out_sel`, `skip_ck40_strobe`, and `clk40_phase[2:0]`.

Table 1: `clock_divider_config` mapping for `test_signal_output`

<code>clock_divider_config</code>	Mode	<code>test_signal_output</code>
2'b00	TDC test	<code>tick40_ce</code>
2'b01	External 40 MHz	<code>clk40</code>
2'b10	BCR pulse	<code>bcr_output</code>
2'b11	External trigger	<code>test_signal_input</code>

The `clk40_phase[2:0]` field is read-only, while `skip_ck40_strobe` is write-only. The functions can be tested by monitoring the test-output signal on an oscilloscope under the different configurations described below.

With the default setting, `clock_divider_config[1:0] = 2'b00`, the test-output signal should be `tick40_ce`, which is a 40 MHz signal with a 12.5% duty cycle.

When `clock_divider_config[1:0]` is set to `2'b01`, the test-output signal should be the 40 MHz clock `clk40`. If an external 40 MHz clock synchronized to the input 320 MHz clock is applied to the test-input port, the value of `clk40_phase[2:0]` indicates the relative phase between `clk40` and the external 40 MHz clock. Each write to `skip_ck40_strobe` shifts the phase of `clk40` by one 3.125 ns step. The corresponding change can be observed both in `clk40_phase[2:0]` and on the oscilloscope. When `clk40_phase[2:0]` is 0, the phase difference between `clk40` and the external clock is at its minimum.

When `clock_divider_config[1:0]` is set to `2'b10`, the test-output signal is the BCR output, which is a single clock-cycle pulse with a repetition period of 512 bunch crossings. When an external BCR signal with a frequency of 40 MHz/512 is applied to the test-input port, the BCR output should align to the phase of the external BCR input.

When `clock_divider_config[1:0]` is set to `2'b11`, the test-output signal is a copy of the external test-input signal, which is used as an external trigger signal.

When `test_out_sel` is set to 1, the test-output port carries the internal 320 MHz clock regardless of the setting of `clock_divider_config[1:0]`.

4 TDC

As shown in Fig. 2, the TDC measures the timing of input `hit` signals and produces a packed 64-bit output word, `tdc_data`, and a hit flag, `hit_flag`, for downstream readout. The TDC can also be tested using the external test input.

The TDC runs on the 320 MHz clock, `clk320`. The `tick40_ce` strobe is asserted for one `clk320` cycle every eight cycles and marks the 40 MHz frame boundary used by the digital pipeline. TDC time measurements are referenced to this boundary.

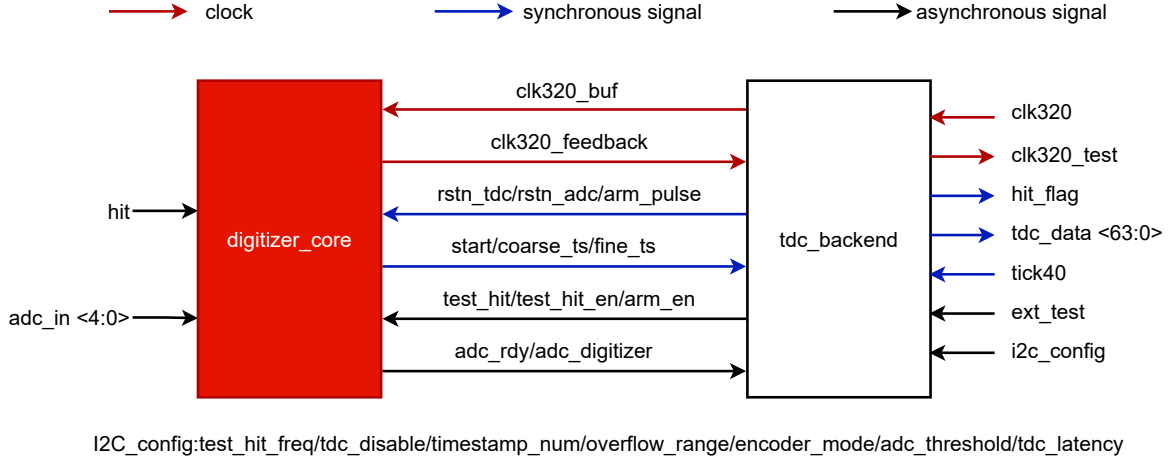


Figure 2: TDC top-level architecture.

The TDC module can be reset through either the I²C reset control or the digital reset input on pin 66, as listed in Table 7 in Section 8.

Each TDC channel can be masked using the I²C register `channel_mask`. Masked channels are ignored. Each channel also has an independent ADC threshold setting. If the ADC value is below the configured threshold, the TDC ignores the hit.

In the TDC configuration, `TDC_timestamp_num` and `tdc_latency` are global configuration registers shared by all TDC channels. The `TDC_timestamp_num` register defines the number of timestamps recorded for each hit. Values from 2 to 65 directly represent the number of timestamps. The encoded values 0 and 1 represent 64 and 65 timestamps, respectively.

The `tdc_latency` register defines the TDC output latency in 40 MHz clock cycles. It should be set large enough to allow the TDC operation to complete. Each additional timestamp requires one more 320 MHz clock cycle to acquire the sample. Setting `tdc_latency` to 10 is safe for all `TDC_timestamp_num` settings. A smaller `tdc_latency` setting reduces the TDC dead time.

The total latency of the TDC digital pipeline depends on the number of timestamps. Expressed in 320 MHz clock cycles, the latency is

$$L_{\text{clk}} = \text{timestamp_num} + 5, \quad (1)$$

where `timestamp_num` is the actual number of acquired timestamps. The additional five cycles are fixed pipeline stages used for data processing.

The register `TDC_overflow_range` defines the maximum allowed deviation from the first calibration code before an overflow condition is reported. This setting depends on the uniformity of the delay line.

The registers `TDC_ext_test` and `test_hit_freq[1:0]` are used for TDC test-hit generation. As shown in Table 1, when `test_hit_freq[1:0]` is set to `2'b00`, no internal test hit is generated. The `TDC_ext_test` register selects the test-hit source: 0 selects the on-chip test-hit generator, and 1 selects the external test input.

The on-chip test-hit generator is mainly intended for readout validation. Its data rate is limited by `test_hit_freq[1:0]` and is designed to stay within the readout bandwidth.

ext_test	test_hit_freq[1:0]	Meaning
1'b1	2'b00	External source; no test hit (off).
1'b1	2'b01	External source; cap = $f_{\text{base}}/128$ (≈ 312.5 kHz @ 40 MHz).
1'b1	2'b10	External source; cap = $f_{\text{base}}/16$ (≈ 2.5 MHz @ 40 MHz); one TDC only.
1'b1	2'b11	External source; no input frequency cap.
1'b0	2'b00	On-chip source; no test hit (off).
1'b0	2'b01	On-chip synchronous source; input cap ≈ 4 Hz.
1'b0	2'b10	On-chip synchronous source; input cap ≈ 60 Hz.
1'b0	2'b11	On-chip synchronous source; input cap ≈ 1 kHz.

When the TDC data is ready, `hit_flag` is asserted. The readout logic packs the 64-bit TDC data word into a data frame for output. As shown in Figure 3, the TDC word format can be selected between raw-data mode and normal mode.

The 64-bit TDC word is divided into two 32-bit words. The upper 32-bit word is the same in both modes. The lower 32-bit word depends on the selected mode. In raw-data mode, the supported number of timestamps is limited to 2 through 9. In normal mode, the supported number of timestamps is 2 through 65.

Normal Mode:

0	ERR_CODE(3b)	CH_ID(4b)	CLK_TAG(3b)	ADC(5b)	TOA0(8b)	CAL0(8b)
0	CAL_SUM(15b)				TOA_SUM(16b)	

Raw Data Mode:

0	ERR_CODE(3b)			CH_ID(4b)	CLK_TAG(3b)	ADC(5b)	TOA0(8b)			CAL0(8b)		
---	--------------	--	--	-----------	-------------	---------	----------	--	--	----------	--	--

0	0	0	0	dCAL1(4b)	dCAL2(4b)	dCAL3(4b)	dCAL4(4b)	dCAL5(4b)	dCAL6(4b)	dCAL7(4b)
---	---	---	---	-----------	-----------	-----------	-----------	-----------	-----------	-----------

Figure 3: 64-bit packing for Normal and Raw data modes (MSB $\rightarrow$ LSB left-to-right).

For each hit, the hit time can be reconstructed from the normal-mode TDC word as follows. Let T_{320} denote the period of the 320 MHz clock:

$$T_{320} = 3125 \text{ ps.} \quad (2)$$

Let n be the actual number of timestamps used for the hit, with $2 \leq n \leq 65$. The triangular weight sum is defined as

$$W = \left\lfloor \frac{n^2}{4} \right\rfloor. \quad (3)$$

Table 2: TDC data-word definition.

Mode	Field	Name / Value	Description
Common	Data1[31]	1'b0	Data-word identifier or reserved leading bit.
Common	Data1[30:28]	Err_Code	3-bit error-code field.
Common	Data1[27:24]	CH_ID	4-bit channel identifier.
Common	Data1[23:21]	CLK_TAG	3-bit clock tag, showing which clk320 cycle within the 8-cycle tick40_ce frame the hit occurred
Common	Data1[20:16]	ADC	5-bit ADC value.
Common	Data1[15:8]	TOA0	8 bits fine Time-of-Arrival code from the first timestamp cell.
Common	Data1[7:0]	CAL0	8 bits calibration code from the first fine delay element.
Normal	Data0[31]	1'b0	32-bit LSB word leading bit, fixed at 1'b0.
Normal	Data0[30:16]	CAL_SUM	15 bits Signed accumulation of calibration codes across all timestamps, used to estimate fine bin width.
Normal	Data0[15:0]	TOA_SUM	16 bits signed accumulation of fine time codes; used to compute averaged fine timing.
Raw data	Data0[31:28]	4'b0000	32-bit LSB word leading bit, fixed at 1'b0.
Raw data	Data0[27:24]	dCAL1	4-bit code, CAL1-CAL0.
Raw data	Data0[23:20]	dCAL2	4-bit code, CAL2-CAL0.
Raw data	Data0[19:16]	dCAL3	4-bit code, CAL3-CAL0.
Raw data	Data0[15:12]	dCAL4	4-bit code, CAL4-CAL0.
Raw data	Data0[11:8]	dCAL5	4-bit code, CAL5-CAL0.
Raw data	Data0[7:4]	dCAL6	4-bit code, CAL6-CAL0.
Raw data	Data0[3:0]	dCAL7	4-bit code, CAL7-CAL0.

The average calibration slope is estimated from `CAL0` and `CAL_SUM`:

$$m_{\text{triangle}} = \text{CAL0} + \frac{\text{CAL_SUM}}{W}. \quad (4)$$

The corresponding fine bin size is

$$\Delta t_{\text{fine}} = \frac{T_{320}}{m_{\text{triangle}}}. \quad (5)$$

The average fine-time code is computed as

$$C_{\text{fine}} = \text{TOA0} + \frac{\text{TOA_SUM}}{n} + \frac{\text{CAL0}(n-1)}{2} + 0.5. \quad (6)$$

The fine-time contribution is then

$$t_{\text{fine}} = C_{\text{fine}} \cdot \Delta t_{\text{fine}}. \quad (7)$$

Because the averaged timestamp is centered over n consecutive 320 MHz clock cycles, the center-position correction is

$$t_{\text{shift}} = \frac{n-1}{2} T_{320}. \quad (8)$$

The reconstructed hit time is

$$t_{\text{hit}} = \text{CLK_TAG} \cdot T_{320} + t_{\text{fine}} - t_{\text{shift}}. \quad (9)$$

When the TDC word is in raw-data mode, the individual timestamp samples can be reconstructed explicitly. In this mode, the actual number of timestamps is limited to $2 \leq n \leq 9$. The raw-data word provides `CAL0` and up to seven differential calibration fields, `dCAL1` through `dCAL7`. For n timestamp samples, only the first $n-2$ differential calibration fields are valid. For example, when $n = 9$, all seven differential calibration fields are valid. When $n = 2$, no differential calibration field is used.

For n timestamp samples, there are $n-1$ calibration intervals. Define c_i as the reconstructed calibration interval between timestamp sample i and timestamp sample $i+1$, where $0 \leq i \leq n-2$. The calibration intervals are reconstructed as

$$c_i = \begin{cases} \text{CAL0}, & i = 0, \\ \text{CAL0} + \text{dCAL}_i, & 1 \leq i \leq n-2, \end{cases} \quad (10)$$

where `dCALi` denotes the corresponding differential calibration field `dCAL1`, `dCAL2`, ..., `dCAL7`. By definition,

$$\text{dCAL}_i = c_i - \text{CAL0}. \quad (11)$$

The cumulative fine-code position of timestamp sample k is

$$S_0 = 0, \quad S_k = \sum_{i=0}^{k-1} c_i, \quad 1 \leq k \leq n-1. \quad (12)$$

The reconstructed fine-code samples are then

$$C_k = \text{TOA0} + S_k + 0.5, \quad 0 \leq k \leq n-1. \quad (13)$$

A linear fit is applied to C_k as a function of the timestamp index k :

$$C_k \approx C_{\text{fit},0} + m_{\text{LR}}k. \quad (14)$$

The linear-regression slope is

$$m_{\text{LR}} = \frac{\sum_{k=0}^{n-1} (k - \bar{k})(C_k - \bar{C})}{\sum_{k=0}^{n-1} (k - \bar{k})^2}, \quad (15)$$

where

$$\bar{k} = \frac{n-1}{2}, \quad \bar{C} = \frac{1}{n} \sum_{k=0}^{n-1} C_k. \quad (16)$$

The fine bin size estimated from the raw-data samples is

$$\Delta t_{\text{fine,LR}} = \frac{T_{320}}{m_{\text{LR}}}. \quad (17)$$

The fitted fine-code value at the first timestamp sample is

$$C_{\text{fit},0} = \bar{C} - m_{\text{LR}}\bar{k}. \quad (18)$$

The reconstructed hit time from the raw-data linear-regression method is

$$t_{\text{hit,raw}} = \text{CLK_TAG} \cdot T_{320} + C_{\text{fit},0} \cdot \Delta t_{\text{fine,LR}}. \quad (19)$$

The 3-bit error code in each TDC word is used for debug and monitoring purpose. The definition of the error code is shown in Table 3.

Table 3: Error code definitions.

Name	Value	Meaning
NO_ERROR	3'b000	No error detected.
CAL_OVERFLOW	3'b001	Calibration delta exceeded <code>overflow_range</code> .
COARSE_PHASE_MISMATCH	3'b010	Inconsistent coarse phase alignment detected.
FINE_PHASE_MISMATCH	3'b011	Inconsistent fine phase alignment detected.

5 I²C Slow Control

The FCFD chip uses an I²C block for slow configuration and monitoring. A clock divider provides a dedicated 40 MHz clock for this block. The I²C block can be reset by the reset pin. This block is not triplicated.

The 7-bit I²C device address has the format 7'b011xxxx. The upper three bits are fixed to 3'b011, while the lower four bits are user-defined by pins 21, 24, 27, and 30, as listed in Table 7.

As shown in Table 4, the register bits in this module have three access types:

- **RW**: Read/write register bits used for configuration. These bits can be written through I²C and read back.
- **RO**: Read-only register bits used for status monitoring.
- **WO**: Write-only register bits. A write operation generates a pulse signal to control other modules. Reading these bits is not meaningful.

Table 4: FCFD I2C register map.

Reg Addr	Field Name	Access Type	Bit Range	Default	Description
0x00	clk_enable	RW	[0]	1	Enable the clock input receiver. The actual control bit is not used. The clock input receiver is always enabled.
	clk_inv_data	RW	[1]	0	Invert the clock polarity.
	clk_en_term	RW	[2]	1	Enable input termination for the clock receiver.
	clk_eq[1:0]	RW	[4:3]	0	The clock receiver equalizer setting.
	clk_common_mode	RW	[5]	0	The clock receiver common-mode setting.
0x01	test_rx_enable	RW	[0]	1	Enable the test input receiver.
	test_rx_inv_data	RW	[1]	0	Invert the test receiver data polarity.
	test_rx_en_term	RW	[2]	1	Enable input termination for the test receiver.
	test_rx_eq[1:0]	RW	[4:3]	0	Select the test receiver equalizer setting.
	test_rx_common	RW	[5]	0	Select the test receiver common-mode setting.
0x03	ser_inv_data	RW	[0]	0	Invert the serializer driver output data polarity.
	ser_drive_str[2:0]	RW	[3:1]	3	Set the serializer driver output driver strength.
	ser_emph_mode[1:0]	RW	[5:4]	0	Select the serializer driver pre-emphasis mode.
	ser_emph_width[1:0]	RW	[7:6]	0	Set the lower bits of the serializer driver pre-emphasis pulse width.
0x04	ser_emph_width[2]	RW	[0]	0	Set the upper bit of the serializer driver pre-emphasis pulse width.
	ser_emph_str[2:0]	RW	[3:1]	3	Set the serializer driver pre-emphasis strength.
	ser_lowSupplyR	RW	[4]	0	Set serializer driver low supply resistance.
0x05	test_tx_inv_data	RW	[0]	0	Invert the test output data polarity.
	test_tx_drive_str[2:0]	RW	[3:1]	3	Set the test output driver strength.
	test_tx_emph_mode[1:0]	RW	[5:4]	0	Select the test output pre-emphasis mode.
<i>Continued on next page</i>					

Reg Addr	Field Name	Access Type	Bit Range	Default	Description
	test_tx_emph_width[1:0]	RW	[7:6]	0	Set the lower bits of the test output pre-emphasis pulse width.
0x06	test_tx_emph_width[2]	RW	[0]	0	Set the upper bit of the test output pre-emphasis pulse width.
	test_tx_emph_str[2:0]	RW	[3:1]	3	Set the test output pre-emphasis strength.
	test_tx_lowSupplyR	RW	[4]	0	Set test out driver low supply resistance.
0x08	disable_scrambler	RW	[0]	0	Disable the data scrambler.
	test_hit	RW	[1]	0	Enable test-hit operation.
	test_pattern_sel[1:0]	RW	[3:2]	0	Select the readout test pattern.
	group_id[2:0]	RW	[6:4]	0	Set the group identifier used in readout data frame.
0x09	ADC_Threshold0[4:0]	RW	[4:0]	0	ADC threshold for channel 0.
0x0A	ADC_Threshold1[4:0]	RW	[4:0]	0	ADC threshold for channel 1.
0x0B	ADC_Threshold2[4:0]	RW	[4:0]	0	ADC threshold for channel 2.
0x0C	ADC_Threshold3[4:0]	RW	[4:0]	0	ADC threshold for channel 3.
0x0D	ADC_Threshold4[4:0]	RW	[4:0]	0	ADC threshold for channel 4.
0x0E	ADC_Threshold5[4:0]	RW	[4:0]	0	ADC threshold for channel 5.
0x10	reset_readout	WO	[1]	N/A	Write strobe to reset the readout logic.
	reset_i2c_readout	WO	[3]	N/A	Write strobe to reset the I2C readout logic.
	skip_ck40_strobe	WO	[4]	N/A	Write strobe to change clk40 phase.
0x11	tdc_latency[3:0]	RW	[3:0]	9	Set the TDC readout latency.
0x12	tdc_timestamp_num[5:0]	RW	[5:0]	8	Set the number of TDC timestamps for each measurement.
	tdc_test_hit_freq[1:0]	RW	[7:6]	0	Select the TDC test-hit frequency.
0x13	tdc_ext_test	RW	[0]	0	Enable external TDC test mode.
	tdc_encoder	RW	[1]	0	Select or enable the TDC encoder mode.
	tdc_overflow_range[4:0]	RW	[6:2]	8	Set the TDC overflow range.
0x14	i2c_readout_enable	RW	[0]	0	Enable I2C readout.
	i2c_readout_acq_mode	RW	[1]	0	I ² C readout acquisition mode. 0: Self-trigger mode. 1: External-trigger mode.

Continued on next page

Reg Addr	Field Name	Access Type	Bit Range	Default	Description
	i2c_readout_ext_test	RW	[2]	0	External test mode selection. 0: Trigger Mode. 1: Rearm Mode.
	test_out_sel	RW	[3]	0	0: <code>clock_divider</code> test output. 1: TDC feedback clock signal from channel 5.
	clock_divider_config[1:0]	RW	[5:4]	0	Configure the clock divider setting.
0x15	channel_mask[5:0]	RW	[5:0]	0	Mask out individual channels in readout. Active High.
0x16-- 0x49	i2c_readout_data [415:0]	RO	[415:0]	0	Read-only I2C readout data buffer. Details see Table 6.
0x4A	clk40_phase[2:0]	RO	[2:0]	0	Read-only CK40 phase status.
0x4B	wb_error_count[7:0]	RO	[7:0]	0	8-bit counter for Wishbone Bus not-acknowledge errors.

6 Fast readout

6.1 Fast Readout Architecture

An overview of the **fast_readout** architecture is shown in Figure 4. The diagram shows the data path from the TDC interface to the serial output and identifies the major functional blocks and their connections. Starting from the six-channel TDC inputs, hit data are evaluated, formatted into event frames, buffered, and then streamed through the serialization path. The readout logic decouples event acceptance from serial transmission: event formation and buffering operate in the 40 MHz clock domain, while the serialized output runs at 320 Mbps with the most significant bit transmitted first.

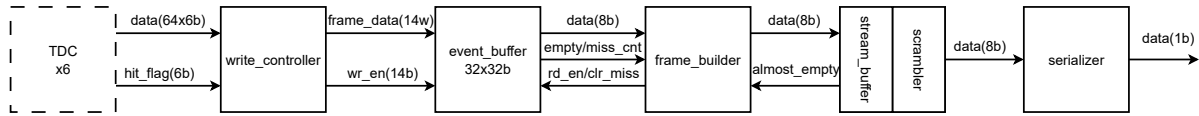


Figure 4: The fast readout diagram

When any TDC channel reports a hit, **fast_readout** evaluates whether the internal buffer has sufficient free space to store the entire event. If enough space is available, the module accepts the event, formats the TDC data into a data frame, and schedules the frame for transmission. If insufficient space is available, the module drops the event and increments a *missing counter*. The missing counter is later reported to the receiver using a filler word.

6.2 Readout data frame

The **fast_readout** module packages Time-to-Digital Converter (TDC) hit data into framed words, and transmits the resulting stream over a serial output link at 320 Mbps. The data frame definition can be seen in Figure 5. The data word definition can be seen in Section 4.

HEADER DELIMITER 0x8B (8b)	TYPE (1b)	GROUPID(3b)	UPPER_BXID(11b)	BXID(9b)
TRAILER DELIMITER 0x8D (8b)	STATUS (8b)		CRC(16b)	
FILLER DELIMITER 0x8E (8b)	MODULE_ID(16)			MISSING EVT COUNTER(8)

Figure 5: The FCFD data frame definition except data word.

fast_readout continues draining the accepted frames from the internal buffer. When the internal buffer becomes empty, the module transmits filler words on the serial link.

If the missing counter is nonzero, its value is reported in the next available filler word and then cleared after the report. To prevent counter overflow, when the missing counter reaches 232 the module forces a filler insertion even if event data are still pending in the internal event buffer. Over time, the total missing count reported in filler words plus the number of received data frames should match the total number of TDC hit events presented to **fast_readout**.

Table 5: Header, trailer, and filler word definition for the **fast_readout** data format.

Field	Name / Value	Description
Header[31:24]	8'h8B	Header delimiter.
Header[23]	Type	Frame type. 0: normal data; 1: test pattern. This value is determined by the I ² C configuration register test_hit .
Header[22:20]	GroupID	3-bit group identifier. This value is determined by the I ² C configuration register group_id [2:0].
Header[19:9]	Upper_BXID	11-bit upper bunch-crossing identifier.
Header[8:0]	BXID	9-bit bunch-crossing identifier, ranging from 0 to 503.
Trailer[31:24]	8'h8D	Trailer delimiter.
Trailer[23:16]	Status	8-bit frame status field.
Trailer[15:0]	CRC	16-bit CRC code.
Filler[31:24]	8'h8E	Filler delimiter.
Filler[23:8]	Module_ID	16-bit module identifier.
Filler[7:0]	MissingEvtCounter	8-bit missing-event counter.

6.3 Link test patterns and scrambling

For serial-link validation, **fast_readout** can be configured to transmit test patterns in place of normal framed data. Supported patterns include an 8-bit counter, a fixed pattern 0x5C, and a PRBS7 sequence.

For any output mode (normal frames or test patterns), the scrambler can be enabled for DC balanced data stream.

- **disable_scrambler** (input): When asserted, the scrambler is bypassed and the selected byte stream is transmitted directly. When deasserted, the output stream is scrambled to improve DC balance.
- **tx_pattern_sel**[1:0] (input): Selects the transmitted byte source before scrambling and serialization:
 - 00: Normal data path (framed event data from the internal stream buffer).
 - 01: PRBS7 test pattern.
 - 10: Fixed 8-bit pattern 0x5C.
 - 11: 8-bit counter, incrementing at 40 MHz.

7 I²C Readout

The I²C readout provides access to the TDC data through the slow-control interface. Because reading the TDC data through I²C takes multiple clock cycles, the TDC data is first latched by a trigger signal and then read out through the I²C registers.

The I²C readout supports two acquisition modes: self-trigger mode and external-trigger mode. The mode is selected by the I²C configuration register `i2c_readout_acq_mode`, as listed in Table 4. In self-trigger mode, the trigger signal is generated by any hit from the TDC channels. In external-trigger mode, the trigger signal is taken from the external test input port, pins 3 and 4, as listed in Table 7.

The clock for the I²C readout block is gated by the control signal `i2c_readout_enable`. This enable signal is set to 0 by default and must be asserted before using the I²C readout.

The I²C readout block can be reset by either the digital reset port or the I²C reset strobe `reset_i2c_readout`. The `reset_i2c_readout` strobe resets only this block and does not affect other circuits. It is also used to clear the latched trigger state before the next data readout.

The captured data is read through the read-only register `i2c_readout_data`, with the data fields defined in Table 6.

In self-trigger mode, both `i2c_readout_acq_mode` and `i2c_readout_ext_test` should be set to zero.

In this mode, the data latch operation can be triggered by a hit flag from any of the six TDC channels. The data buffer stores all TDC data words from the six channels, as well as `hit_trig_bcid[8:0]`, which contains the BCID value at the time of the hit flag that triggered the latch operation. The data buffer is held until `reset_i2c_readout` is issued to clear the triggered state before the next readout.

To use external-trigger mode, the 2-bit configuration signal `clock_divider_config` must be set to route the external trigger signal from the test input port.

When `i2c_readout_acq_mode` is set to 1, the I²C readout operates in external-trigger mode. If `i2c_readout_ext_test` is set to 0, the rising edge of the external trigger signal immediately latches the data for readout. If `i2c_readout_ext_test` is set to 1, the rising edge of the external trigger signal arms the I²C readout block. The block then waits for the first hit flag to latch the TDC data for readout.

The BCID is latched to identify the hit timing. In external-trigger mode with rearm mode, both `ext_trig_bcid[8:0]` and `hit_trig_bcid[8:0]` are valid for timing analysis.

In both modes, the user needs to check `TDC_hit_flag[5:0]`. When this register is not all zero, the data is ready for readout. Only the TDC words corresponding to the non-zero hit flags need to be read. After the readout is complete, the user should issue `reset_i2c_readout` to clear the triggered state and prepare the block for the next readout operation.

Table 6: I²C readout data buffer field mapping. The six TDC data words are byte-addressed, with the least significant byte stored at the lowest address.

Reg Addr	Field Name	Bit Range	Description
0x16	TDC_hit_flag[5:0]	[5:0]	Hit flags for TDC channels 0 through 5. Bit i corresponds to TDC channel i .
0x16	self_trig_flag	[6]	Indicates that data was captured by a self-trigger event.
0x16	ext_trig_flag	[7]	Indicates that data was captured by an external-trigger event.
0x17--0x1E	ch0_TDC_data[63:0]	[7:0] per address	TDC data for channel 0. Address 0x17 contains bits [7:0], and address 0x1E contains bits [63:56].
0x1F--0x26	ch1_TDC_data[63:0]	[7:0] per address	TDC data for channel 1. Address 0x1F contains bits [7:0], and address 0x26 contains bits [63:56].
0x27--0x2E	ch2_TDC_data[63:0]	[7:0] per address	TDC data for channel 2. Address 0x27 contains bits [7:0], and address 0x2E contains bits [63:56].
0x2F--0x36	ch3_TDC_data[63:0]	[7:0] per address	TDC data for channel 3. Address 0x2F contains bits [7:0], and address 0x36 contains bits [63:56].
0x37--0x3E	ch4_TDC_data[63:0]	[7:0] per address	TDC data for channel 4. Address 0x37 contains bits [7:0], and address 0x3E contains bits [63:56].
0x3F--0x46	ch5_TDC_data[63:0]	[7:0] per address	TDC data for channel 5. Address 0x3F contains bits [7:0], and address 0x46 contains bits [63:56].
0x47--0x48	hit_trig_bcid[8:0]	0x48[0], 0x47[7:0]	9-bit BCID associated with the hit flag.
0x48--0x49	ext_trig_bcid[8:0]	0x49[1:0], 0x48[7:1]	9-bit BCID associated with the ext-trigger.

8 Pinout

8.1 pin name and map

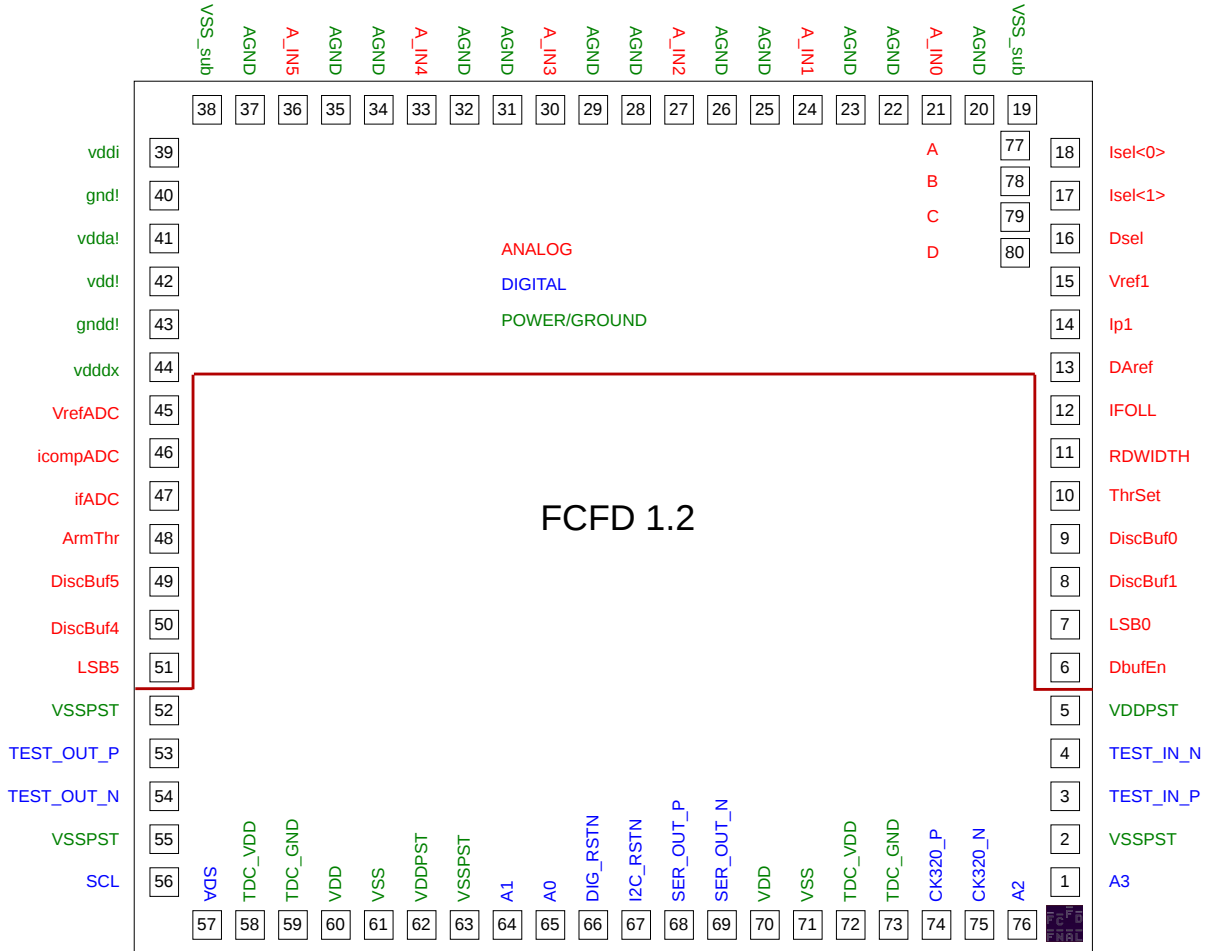


Figure 6: FCFD1v2 pin number and name map.

8.2 Pinout table

Table 7 summarizes the pads of the main ETROC2.

Table 7: FCFD 1.2 Pinout (Preliminary)

Pin #	Internal Name	External Name	Pad Center [μm]	Pad Size [μm]	Description
1	A3	A3	(3223,225)	100×85	I ² C address bit 3
2	VSSPST	VSSPST	(3223,375)	100×85	ESD ground
3	TEST_IN_P	TEST_IN_P	(3223,525)	100×85	Differential test signal input (+)
4	TEST_IN_N	TEST_IN_N	(3223,675)	100×85	Differential test signal input (-)
5	VDDPST	VDDPST	(3223,825)	100×85	ESD supply 1.2 V

Continued on next page

Pin #	Internal Name	External Name	Pad Center [μm]	Pad Size [μm]	Description
6	DbufEn	DbufEn	(3229.5,967.8)	86×86	Discriminator buffer enable
7	LSB0	LSB0	(3229.5,1117.8)	86×86	LSB control bit 0
8	DiscBuf1	DiscBuf1	(3229.5,1267.8)	86×86	Discriminator buffer output 1
9	DiscBuf0	DiscBuf0	(3229.5,1417.8)	86×86	Discriminator buffer output 0
10	ThrSet	ThrSet	(3229.5,1567.8)	86×86	Threshold setting control
11	RDWIDTH	RDWIDTH	(3229.5,1717.8)	86×86	Readout width configuration
12	IFOLL	IFOLL	(3229.5,1867.8)	86×86	Analog bias current
13	DAref	DAref	(3229.5,2017.8)	86×86	Analog reference
14	Ip1	Ip1	(3229.5,2167.8)	86×86	Bias current input
15	Vref1	Vref1	(3229.5,2317.8)	86×86	Reference voltage
16	Dsel	Dsel	(3229.5,2467.8)	86×86	Digital select control
17	Isel[1]	Isel[1]	(3229.5,2617.8)	86×86	Current selection bit 1
18	Isel[0]	Isel[0]	(3229.5,2767.8)	86×86	Current selection bit 0
19	VSS_sub	VSS_sub	(3119.8,2929.5)	66×86	Substrate ground
20	AGND	AGND	(2947.2,2929.5)	66×86	Analog ground
21	A_IN0	A_IN0	(2777.2,2929.5)	66×86	Analog input channel 0
22	AGND	AGND	(2607.2,2929.5)	66×86	Analog ground
23	AGND	AGND	(2497.2,2929.5)	66×86	Analog ground
24	A_IN1	A_IN1	(2327.2,2929.5)	66×86	Analog input channel 1
25	AGND	AGND	(2157.2,2929.5)	66×86	Analog ground
26	AGND	AGND	(2047.2,2929.5)	66×86	Analog ground
27	A_IN2	A_IN2	(1877.2,2929.5)	66×86	Analog input channel 2
28	AGND	AGND	(1707.2,2929.5)	66×86	Analog ground
29	AGND	AGND	(1597.2,2929.5)	66×86	Analog ground
30	A_IN3	A_IN3	(1427.2,2929.5)	66×86	Analog input channel 3
31	AGND	AGND	(1257.2,2929.5)	66×86	Analog ground
32	AGND	AGND	(1147.2,2929.5)	66×86	Analog ground
33	A_IN4	A_IN4	(977.2,2929.5)	66×86	Analog input channel 4
34	AGND	AGND	(807.2,2929.5)	66×86	Analog ground
35	AGND	AGND	(697.2,2929.5)	66×86	Analog ground
36	A_IN5	A_IN5	(527.2,2929.5)	66×86	Analog input channel 5
37	AGND	AGND	(357.2,2929.5)	66×86	Analog ground
38	VSS_sub	VSS_sub	(180.2,2929.5)	66×86	Analog substrate ground
39	vddi	vddi	(70.5,2767.8)	86×86	Frontend digital IO supply
40	gnd!	gnd!	(70.5,2617.8)	86×86	Frontend ground
41	vdda!	vdda!	(70.5,2467.8)	86×86	Analog supply
42	vdd!	vdd!	(70.5,2317.8)	86×86	Core supply
43	gndd!	gndd!	(70.5,2167.8)	86×86	Digital ground
44	vdddx	vdddx	(70.5,2017.8)	86×86	Digital supply
45	VrefADC	VrefADC	(70.5,1867.8)	86×86	ADC reference voltage
46	icompADC	icompADC	(70.5,1717.8)	86×86	ADC comparator bias current
47	ifADC	ifADC	(70.5,1567.8)	86×86	ADC bias current
48	ArmThr	ArmThr	(70.5,1417.8)	86×86	Arming threshold control
49	DiscBuf5	DiscBuf5	(70.5,1267.8)	86×86	Discriminator buffer output 5
50	DiscBuf4	DiscBuf4	(70.5,1117.8)	86×86	Discriminator buffer output 4
51	LSB5	LSB5	(70.5,967.8)	86×86	LSB control bit 5
52	VSSPST	VSSPST	(77,825)	100×85	ESD ground
53	TEST_OUT_P	TEST_OUT_P	(77,675)	100×85	Differential test output (+)
54	TEST_OUT_N	TEST_OUT_N	(77,525)	100×85	Differential test output (-)
55	VSSPST	VSSPST	(77,375)	100×85	ESD ground
56	SCL	SCL	(77,225)	100×85	I ² C clock
57	SDA	SDA	(225,77)	85×100	I ² C data
58	TDC_VDD	TDC_VDD	(375,77)	85×100	TDC power supply
59	TDC_GND	TDC_GND	(525,77)	85×100	TDC ground

Continued on next page

Pin #	Internal Name	External Name	Pad Center [μm]	Pad Size [μm]	Description
60	VDD	VDD	(675,77)	85×100	Digital core supply
61	VSS	VSS	(825,77)	85×100	Digital ground
62	VDDPST	VDDPST	(975,77)	85×100	ESD supply
63	VSSPST	VSSPST	(1125,77)	85×100	ESD ground
64	A1	A1	(1275,77)	85×100	I ² C address bit 1
65	A0	A0	(1425,77)	85×100	I ² C address bit 0
66	DIG_RSTN	DIG_RSTN	(1575,77)	85×100	Digital reset, active low
67	I2C_RSTN	I2C_RSTN	(1725,77)	85×100	I ² C reset, active low
68	SER_OUT_P	SER_OUT_P	(1875,77)	85×100	Serial data output (+)
69	SER_OUT_N	SER_OUT_N	(2025,77)	85×100	Serial data output (-)
70	VDD	VDD	(2175,77)	85×100	Digital core supply
71	VSS	VSS	(2325,77)	85×100	Digital ground
72	TDC_VDD	TDC_VDD	(2475,77)	85×100	TDC power supply
73	TDC_GND	TDC_GND	(2625,77)	85×100	TDC ground
74	CK320_P	CK320_P	(2775,77)	85×100	320 MHz clock input (+)
75	CK320_N	CK320_N	(2925,77)	85×100	320 MHz clock input (-)
76	A2	A2	(3075,77)	85×100	I ² C address bit 2
77	A	A	(3059.8,2812.8)	86×66	
78	B	B	(3059.8,2706.8)	86×66	
79	C	C	(3059.8,2600.8)	86×66	
80	D	D	(3059.8,2494.8)	86×66	

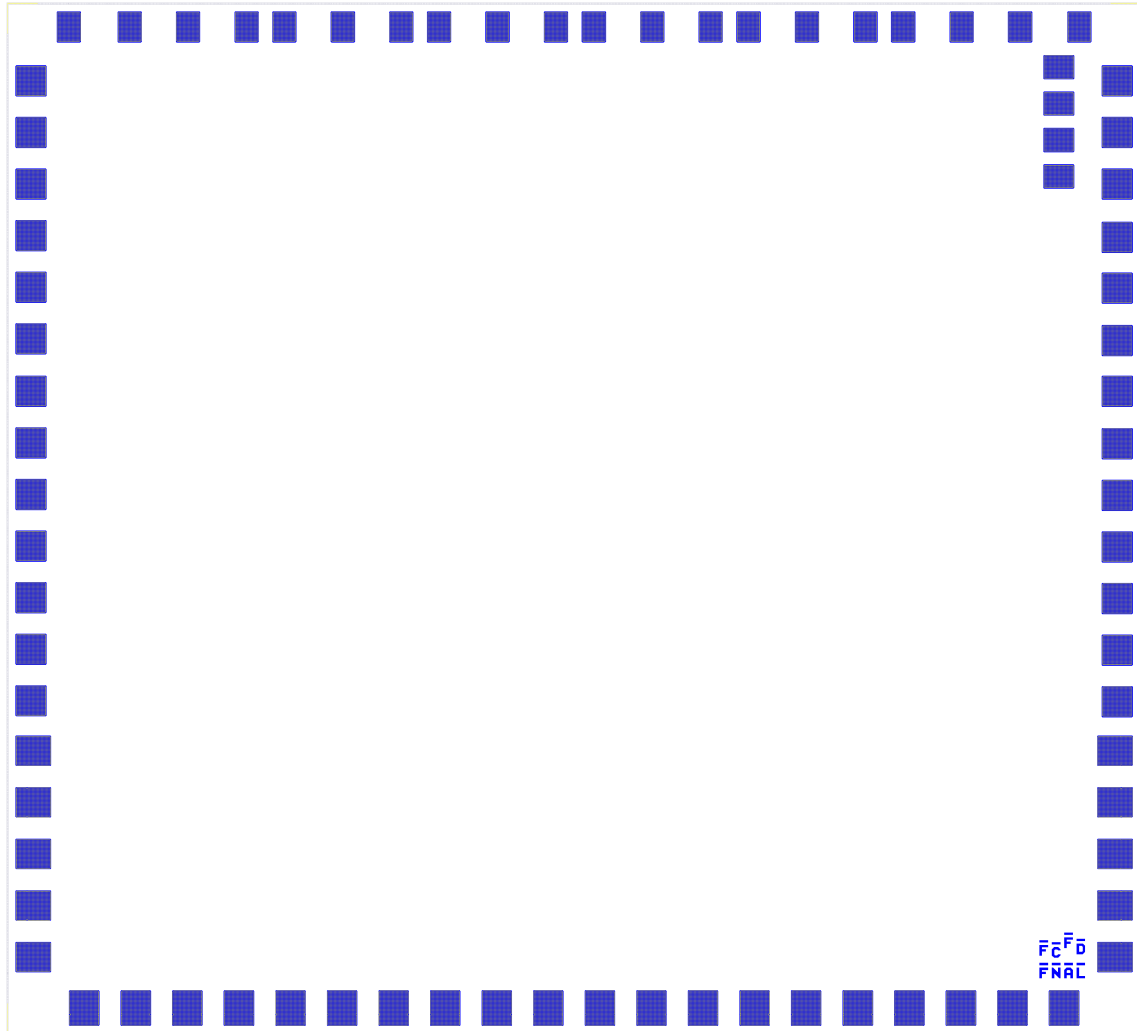


Figure 7: FCFD1v2 pads actual distribution. Die size is $3000 \mu m(H) \times 3300 \mu m(W)$